ECE 1774 2nd Project

jdr129

April 2026

1 Project Overview

This report documents the design and implementation of a Python-based power system analysis tool developed across nine milestones. The simulator is built around a five-bus reference network and is capable of performing Newton–Raphson power flow analysis and symmetrical three-phase fault analysis, with all results validated against PowerWorld Simulator and standard textbook references. The implementation follows an incremental, object-oriented approach in which each milestone extends the capability of the previous, progressing from basic data structures to a fully functional numerical solver.

The foundational milestones established the core equipment classes— `Bus`, `Transformer`, `TransmissionLine`, `Load`, and `Generator`—along with a `Circuit` container and a `Settings` class for system-wide per-unit parameters. These components were then used to construct the per-unit primitive admittance matrices (Y_{prim}) and the full nodal admittance matrix (Y_{bus}), which serves as the mathematical foundation for all subsequent analyses. Bus voltage state variables, power injection equations, and the power mismatch vector were subsequently implemented to prepare the system for iterative solution.

The numerical core of the simulator consists of a Jacobian matrix construction module and a Newton–Raphson solver that iteratively updates bus voltage magnitudes and angles until convergence is achieved. The solver was then extended to support symmetrical fault analysis, incorporating generator subtransient reactance X'' into a modified Y_{bus}, inverting it to form the bus impedance matrix Z_{bus}, and computing subtransient fault currents and post-fault bus voltages. All analyses are conducted on a per-unit basis with a system base of $S_{\text{base}} = 100$ MVA.

2 Class Diagrams

The simulator is implemented using multiple object-oriented classes that represent the fundamental components of a power system, including buses, generators, loads, transformers, transmission lines, and the overall circuit structure. Each class contains attributes and methods that define the electrical properties

and behaviors of the corresponding component, allowing the full power system model to be constructed and analyzed programmatically.

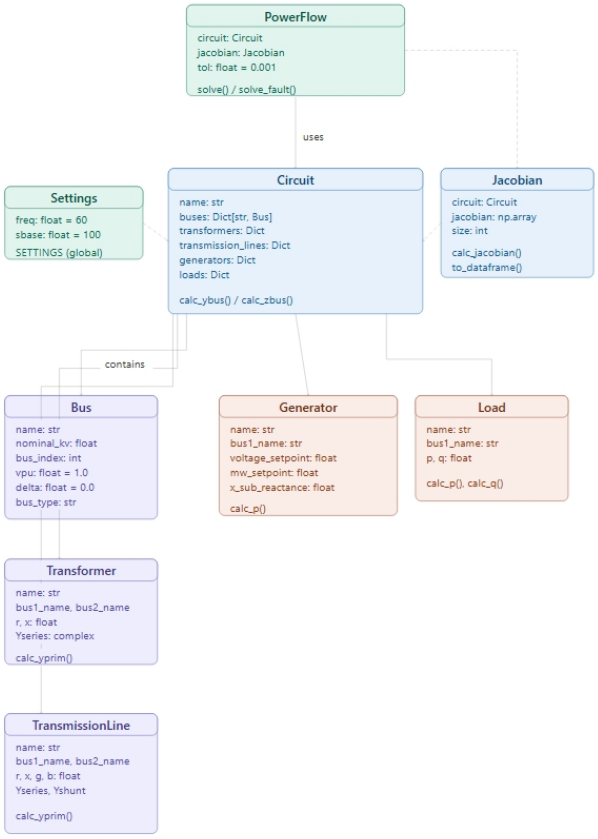

Figure 1: UML class diagram of the power system simulator showing all classes, their attributes and methods, and the containment and dependency relationships between them.

Figure 1 shows the class structure used to model the power system simulator. The diagram illustrates the relationships between buses, circuit elements, and supporting classes used in the implementation. The `PowerFlow` class sits at the top of the hierarchy and serves as the primary solver, holding references to both the `Circuit` and `Jacobian` classes. The `Circuit` class acts as the central container, aggregating all network components — `Bus`, `Transformer`, `TransmissionLine`, `Generator`, and `Load` — and providing the methods required to assemble Y_{bus}, compute power injections, and construct Z_{bus} for fault analysis. The `Jacobian` class operates alongside `Circuit` to provide the linearized system matrix required at each Newton–Raphson iteration. The global

`Settings` instance supplies system-wide per-unit base parameters referenced by all equipment classes.

3 Class Description

3.1 Bus Class

The `Bus` class represents an electrical node in the power system network where equipment such as generators, loads, transformers, and transmission lines connect. Each bus serves as a reference point for storing and updating the voltage state variables required during Newton–Raphson power flow analysis.

3.1.1 Attributes

- `name`: A string identifier used to label the bus (e.g., "Bus 1" or "Bus 2").
- `nominal_kv`: A floating-point value representing the nominal voltage of the bus in kilovolts.
- `bus_index`: A unique integer index automatically assigned to each bus instance using a class-level counter. This index is used when constructing network matrices such as Y_{bus}.
- `vpu`: A floating-point variable representing the per-unit voltage magnitude at the bus. Initialized to 1.0 p.u. and updated at each Newton–Raphson iteration.
- `delta`: A floating-point variable representing the voltage phase angle at the bus in degrees. Initialized to 0.0° for a flat start.
- `bus_type`: A string classification of the bus. Valid types are `Slack`, `PV`, and `PQ`. The default type is `PQ`. An invalid type raises a `ValueError`.

3.1.2 Methods

- `__repr__()`: Returns a string representation of the bus displaying its nominal voltage, used for inspection and debugging.
- `bus_type` (property getter): Returns the bus type currently assigned stored in the private attribute `_bus_type`.
- `bus_type` (property setter): Validates and assigns a new bus type. If the provided type is not a member of `VALID_TYPES` = {`Slack`, `PV`, `PQ`}, a `ValueError` is raised to enforce data integrity.

3.1.3 Role in the Simulator

The `Bus` class provides the foundational node structure for the entire power system model. By storing per-unit voltage magnitude and phase angle as state variables, it directly supports the iterative Newton–Raphson solution process. The enforced bus type classification ensures that the solver correctly handles each bus according to its role: the `Slack` bus fixes voltage magnitude and angle as reference, `PV` buses maintain a specified voltage magnitude with free angle, and `PQ` buses allow both voltage magnitude and angle to be updated during each iteration.

3.2 Generator Class

The `Generator` class represents a synchronous generator connected to a bus in the power system. It models both the real power injection at its terminal bus for power flow analysis and the subtransient reactance required for symmetrical fault analysis.

3.2.1 Attributes

- `name`: A string identifier used to label the generator (e.g., "G1").
- `bus1_name`: A string specifying the name of the bus to which the generator is connected.
- `voltage_setpoint`: A floating-point value representing the generator's terminal voltage setpoint in per-unit. This value is used to enforce the voltage magnitude at `PV` buses.
- `mw_setpoint`: A floating-point value representing the generator's real power output setpoint in megawatts.
- `p`: A floating-point value representing the per-unit real power injection. Computed automatically at initialization by `calc_p()`.
- `x_sub_reactance`: A floating-point value representing the generator's subtransient reactance X'' in per-unit on the generator base. This attribute is used when constructing the modified Y_{bus} for fault analysis.

3.2.2 Methods

- `calc_p()`: Computes and returns the per-unit real power injection by dividing the megawatt setpoint by the system base apparent power `SETTINGS.sbase`. The result is stored in `self.p` upon initialization.

3.2.3 Role in the Simulator

The `Generator` class serves two distinct roles depending on the analysis mode. During power flow analysis, it provides the scheduled real power injection and voltage setpoint used to initialize and constrain the Newton–Raphson iterative solution at `PV` buses. During fault analysis, the subtransient reactance X'' is incorporated into the system admittance matrix to accurately model the generator's transient behavior, enabling computation of the subtransient fault current and post-fault bus voltages.

3.3 Transformer Class

The `Transformer` class represents a two-winding transformer connecting two buses in the power system. It models the transformer using its series impedance in per-unit and computes the primitive admittance matrix Y_{prim} used in the assembly of the system nodal admittance matrix Y_{bus}.

3.3.1 Attributes

- `name`: A string identifier used to label the transformer (e.g., "T1").
- `bus1_name`: A string specifying the name of the first bus to which the transformer is connected.
- `bus2_name`: A string specifying the name of the second bus to which the transformer is connected.
- `r`: A floating-point value representing the per-unit series resistance of the transformer.
- `x`: A floating-point value representing the per-unit series reactance of the transformer.
- `Yseries`: A complex floating-point value representing the series admittance of the transformer, computed at initialization as:

$$Y_{\text{series}} = \frac{1}{r + jx} \tag{1}$$

3.3.2 Methods

- `calc_yprim()`: Constructs and returns the 2×2 primitive admittance matrix for the transformer as a `pandas.DataFrame` labeled by bus names. The matrix is defined as:

$$Y_{\text{prim}} = \begin{bmatrix} Y_{\text{series}} & -Y_{\text{series}} \\ -Y_{\text{series}} & Y_{\text{series}} \end{bmatrix} \tag{2}$$

  Row and column indices correspond to `bus1_name` and `bus2_name`, ensuring unambiguous stamping into Y_{bus}.

3.3.3 Role in the Simulator

The `Transformer` class provides the per-unit primitive admittance matrix that is stamped into the system Y_{bus} during network assembly. By returning Y_{prim} as a labeled `pandas.DataFrame`, the class allows the `Circuit` container to correctly identify the corresponding bus indices and accumulate self- and mutual-admittance contributions without indexing errors. The transformer is modeled purely as a series element, with no shunt branch, consistent with the per-unit formulation used throughout the simulator.

3.4 Transmission Line Class

The `TransmissionLine` class represents a transmission line connecting two buses in the power system using the nominal pi-model. It computes both the series and shunt admittances from per-unit parameters and assembles the 2×2 primitive admittance matrix Y_{prim} used in the construction of Y_{bus}.

3.4.1 Attributes

- `name`: A string identifier used to label the transmission line (e.g., "Line 1").
- `bus1_name`: A string specifying the name of the first bus to which the transmission line is connected.
- `bus2_name`: A string specifying the name of the second bus to which the transmission line is connected.
- `r`: A floating-point value representing the per-unit series resistance of the transmission line.
- `x`: A floating-point value representing the per-unit series reactance of the transmission line.
- `g`: A floating-point value representing the per-unit shunt conductance of the transmission line.
- `b`: A floating-point value representing the per-unit shunt susceptance of the transmission line.
- `Yseries`: A complex floating-point value representing the series admittance, computed at initialization as:

$$Y_{\text{series}} = \frac{1}{r + jx} \tag{3}$$

- `Yshunt`: A complex floating-point value representing the total shunt admittance of the line, computed at initialization as:

$$Y_{\text{shunt}} = g + jb \tag{4}$$

3.4.2 Methods

- `calc_yprim()`: Constructs and returns the 2×2 primitive admittance matrix for the transmission line as a `pandas.DataFrame` labeled by bus names. Using the pi-model, each diagonal entry includes half the shunt admittance in addition to the series admittance, while the off-diagonal entries contain only the negative series admittance:

$$Y_{\text{prim}} = \begin{bmatrix} Y_{\text{series}} + \dfrac{Y_{\text{shunt}}}{2} & -Y_{\text{series}} \\ -Y_{\text{series}} & Y_{\text{series}} + \dfrac{Y_{\text{shunt}}}{2} \end{bmatrix} \tag{5}$$

3.4.3 Role in the Simulator

The `TransmissionLine` class extends the network modeling capability of the simulator beyond ideal transformers by incorporating the shunt charging admittance inherent to physical transmission lines. The pi-model representation distributes half of the total shunt admittance to each terminal bus, which is reflected in the diagonal entries of Y_{prim}. This matrix is subsequently stamped into the system Y_{bus} by the `Circuit` class, ensuring that line charging effects are accurately captured in the power flow and fault analysis formulations

3.5 Settings Class

The `Settings` class defines the system-wide per-unit base parameters used consistently across all equipment classes and numerical solvers in the simulator. A single global instance, `SETTINGS`, is instantiated at module level and accessed throughout the codebase to ensure uniform per-unit scaling.

3.5.1 Attributes

- `freq`: A floating-point value representing the system frequency in hertz. Initialized to a default value of 60 Hz.
- `sbase`: A floating-point value representing the system base apparent power in megavolt-amperes. Initialized to a default value of 100 MVA.

3.5.2 Methods

The `Settings` class contains no methods beyond the constructor. All functionality is provided through direct attribute access on the global `SETTINGS` instance.

3.5.3 Role in the Simulator

The `Settings` class establishes the per-unit reference frame for the entire simulation. By centralizing the system base parameters in a single globally ac-

cessible object, the simulator avoids hardcoded constants scattered across individual classes. Equipment classes such as `Generator` and `Load` reference `SETTINGS.sbase` directly when computing per-unit power injections, ensuring that any change to the system base propagates consistently throughout the model without requiring modifications to individual component definitions.

3.6 Circuit Class

The `Circuit` class serves as the central container for the entire power system model. It stores all network components, assembles the system admittance matrix, computes power injections and mismatches for the Newton–Raphson solver, and constructs the modified admittance and impedance matrices required for fault analysis.

3.6.1 Attributes

- `name`: A string identifier used to label the circuit.
- `buses`: A dictionary mapping bus name strings to `Bus` objects representing all nodes in the network.
- `transformers`: A dictionary mapping transformer name strings to `Transformer` objects in the network.
- `transmission_lines`: A dictionary mapping transmission line name strings to `TransmissionLine` objects in the network.
- `generators`: A dictionary mapping generator name strings to `Generator` objects connected to the network.
- `loads`: A dictionary mapping load name strings to `Load` objects connected to the network.
- `ybus`: A `pandas.DataFrame` representing the system nodal admittance matrix Y_{bus}, indexed by bus names. Initialized to `None` and populated by `calc_ybus()`.
- `zbus`: A `pandas.DataFrame` representing the system bus impedance matrix Z_{bus}, indexed by bus names. Populated by `calc_zbus()` during fault analysis.

3.6.2 Methods

- `duplicate_name(d, name, equipment_type)`: A static method that raises a `ValueError` if a component name already exists in the provided dictionary, preventing duplicate entries in the network model.

- `add_bus()`, `add_transformer()`, `add_transmission_line()`, `add_generator()`, `add_load()`: Factory methods that instantiate the corresponding equipment object, check for duplicate names, store the object in its respective dictionary, and return the created instance.

- `calc_ybus()`: Assembles the $N \times N$ system nodal admittance matrix by iterating over all transformers and transmission lines, retrieving each element's Y_{prim} matrix, and stamping the entries into the corresponding bus index positions. The result is stored as a labeled `pandas.DataFrame` in `self.ybus`.

- `compute_power_injection(bus)`: Computes and returns the calculated real and reactive power injections (P_i, Q_i) at a given bus using the power injection equations:

$$P_i = |V_i| \sum_{j=1}^{N} |V_j| \left(G_{ij} \cos \delta_{ij} + B_{ij} \sin \delta_{ij}\right) \tag{6}$$

$$Q_i = |V_i| \sum_{j=1}^{N} |V_j| \left(G_{ij} \sin \delta_{ij} - B_{ij} \cos \delta_{ij}\right) \tag{7}$$

  where $\delta_{ij} = \delta_i - \delta_j$, and G_{ij} and B_{ij} are the real and imaginary parts of Y_{ij}, respectively.

- `compute_power_mismatch()`: Iterates over all non-slack buses and computes the power mismatch vector by subtracting the calculated power injections from the scheduled values. Real power mismatches ΔP are computed for all `PQ` and `PV` buses, while reactive power mismatches ΔQ are computed for `PQ` buses only. The combined mismatch vector is returned as a `numpy` array.

- `calc_ybus_fault()`: Constructs the modified Y_{bus} for fault analysis by first calling `calc_ybus()` and then adding each generator's subtransient admittance $y'' = 1/(jX'')$ to the diagonal entry corresponding to its terminal bus.

- `calc_zbus()`: Inverts the faulted Y_{bus} using `numpy.linalg.inv()` to obtain the bus impedance matrix:

$$Z_{\text{bus}} = Y_{\text{bus}}^{-1} \tag{8}$$

  The result is stored as a labeled `pandas.DataFrame` in `self.zbus` and returned.

3.6.3 Role in the Simulator

The `Circuit` class is the central integration point of the simulator. It coordinates all network components and provides the mathematical infrastructure

required by both the Newton–Raphson power flow solver and the symmetrical fault analysis module. By encapsulating Y_{bus} assembly, power injection computation, mismatch calculation, and Z_{bus} construction within a single container, the `Circuit` class ensures that all analyses operate on a consistent and fully validated network model.

3.7 Jacobian Class

The `Jacobian` class constructs the Jacobian matrix used in each iteration of the Newton–Raphson power flow solver. The matrix is partitioned into four submatrices representing the partial derivatives of the real and reactive power injections with respect to voltage angles and magnitudes across all non-slack buses.

3.7.1 Attributes

- `circuit`: A reference to the `Circuit` object from which bus and admittance data are retrieved.
- `buses`: The dictionary of `Bus` objects inherited from the associated `Circuit` instance.
- `ybus`: The system nodal admittance matrix Y_{bus} inherited from the associated `Circuit` instance.
- `ordered_buses`: A list of all `Bus` objects sorted in ascending order by `bus_index`, ensuring consistent row and column alignment with Y_{bus}.
- `angle_buses`: A list of all non-slack buses for which voltage angle corrections $\Delta\delta$ are computed. These buses correspond to the columns and rows of submatrices J_1 and J_2.
- `voltage_buses`: A list of all `PQ` buses for which voltage magnitude corrections $\Delta|V|$ are computed. These buses correspond to the columns and rows of submatrices J_3 and J_4.
- `num_buses`: The total number of buses in the network.
- `num_pv`: The number of `PV` buses in the network.
- `size`: The dimension of the square Jacobian matrix, computed as:

$$\text{size} = 2N - 2 - N_{PV} \tag{9}$$

  where N is the total number of buses and N_{PV} is the number of `PV` buses.
- `jacobian`: A `numpy` array of shape (size $\times$ size) storing the assembled Jacobian matrix. Initialized to zeros and populated by `calc_jacobian()`.

- `p_row_map`, `q_row_map`: Dictionaries mapping bus names to their corresponding row indices in the Jacobian matrix for ΔP and ΔQ rows, respectively.
- `delta_col_map`, `v_col_map`: Dictionaries mapping bus names to their corresponding column indices in the Jacobian matrix for δ and $|V|$ columns, respectively.

3.7.2 Methods

- `calc_jacobian(buses, ybus, angles, voltages)`: Computes and returns the full Jacobian matrix by iterating over all non-slack buses and populating the four submatrices according to the following partial derivative expressions.

  The Jacobian is structured as:

$$J = \begin{bmatrix} J_1 & J_2 \\ J_3 & J_4 \end{bmatrix} = \begin{bmatrix} \dfrac{\partial P}{\partial \delta} & \dfrac{\partial P}{\partial |V|} \\ \dfrac{\partial Q}{\partial \delta} & \dfrac{\partial Q}{\partial |V|} \end{bmatrix} \tag{10}$$

  The off-diagonal and diagonal entries of each submatrix are computed as follows. For $J_1 = \partial P / \partial \delta$:

$$J_{1,kk} = -Q_k - B_{kk} V_k^2, \qquad J_{1,kn} = V_k V_n \left(G_{kn} \sin \delta_{kn} - B_{kn} \cos \delta_{kn} \right) \tag{11}$$

  For $J_2 = \partial P / \partial |V|$:

$$J_{2,kk} = \frac{P_k}{V_k} + G_{kk} V_k, \qquad J_{2,kn} = V_k \left(G_{kn} \cos \delta_{kn} + B_{kn} \sin \delta_{kn} \right) \tag{12}$$

  For $J_3 = \partial Q / \partial \delta$:

$$J_{3,kk} = P_k - G_{kk} V_k^2, \qquad J_{3,kn} = -V_k V_n \left(G_{kn} \cos \delta_{kn} + B_{kn} \sin \delta_{kn} \right) \tag{13}$$

  For $J_4 = \partial Q / \partial |V|$:

$$J_{4,kk} = \frac{Q_k}{V_k} - B_{kk} V_k, \qquad J_{4,kn} = V_k \left(G_{kn} \sin \delta_{kn} - B_{kn} \cos \delta_{kn} \right) \tag{14}$$

  Rows corresponding to ΔQ are only populated for `PQ` buses. The `Slack` bus is excluded entirely from all rows and columns.

- `to_dataframe()`: Constructs and returns a `pandas.DataFrame` representation of the Jacobian matrix with descriptive row labels of the form `P Bus` k and `Q Bus` k, and column labels of the form δ `Bus` k and `V Bus` k. Used for inspection and validation of the assembled matrix.
- `print_dataframe(decimals)`: Prints the formatted Jacobian `DataFrame` rounded to the specified number of decimal places. The default precision is two decimal places.

3.7.3 Role in the Simulator

The `Jacobian` class provides the linearization of the nonlinear power flow equations at each Newton–Raphson iteration. By maintaining explicit row and column index maps keyed by bus name, the class ensures that each partial derivative is placed in the correct position relative to the mismatch vector produced by `compute_power_mismatch()`. The formatting methods `to_dataframe()` and `print_dataframe()` support validation by presenting the assembled matrix in a labeled tabular format that can be directly compared against reference values from PowerWorld or textbook examples.

3.8 PowerFlow Class

The `PowerFlow` class serves as the top-level solver for the simulator, integrating the Newton–Raphson power flow algorithm and the symmetrical fault analysis procedure into a unified interface. It accepts a mode parameter at instantiation to determine which analysis is performed when the solver is invoked.

3.8.1 Attributes

- `circuit`: A reference to the `Circuit` object containing the full network model, admittance matrices, and bus state variables.
- `jacobian`: A reference to the `Jacobian` object used to compute the Jacobian matrix at each Newton–Raphson iteration.
- `tol`: A floating-point value specifying the convergence tolerance for the Newton–Raphson solver. The default value is 1×10^{-3} per-unit.
- `max_iter`: An integer specifying the maximum number of Newton–Raphson iterations permitted before a non-convergence error is raised. The default value is 50.
- `mode`: A string specifying the analysis mode. Valid values are `power_flow` and `fault`. An invalid mode raises a `ValueError`.
- `converged`: A boolean flag set to `True` upon successful convergence of the Newton–Raphson solver. Populated during `solve()`.
- `iteration`: An integer recording the number of Newton–Raphson iterations completed before convergence or termination.

3.8.2 Methods

- `run_type(**kwargs)`: Dispatches execution to the appropriate solver method based on `self.mode`. Calls `solve()` for power flow mode and `solve_fault()` for fault mode.
- `solve(tol, max_iter)`: Executes the Newton–Raphson power flow algorithm. The method proceeds as follows:

1. Assembles Y_{bus} by calling `calc_ybus()`.
2. Initializes bus state variables using a flat start: all voltage angles are set to 0.0°, all `PQ` bus voltage magnitudes are set to 1.0 p.u., and `PV` bus voltage magnitudes are set to their generator voltage setpoints.
3. At each iteration, computes the power mismatch vector f and the Jacobian matrix J, then solves the linear correction system:

$$J \cdot \Delta x = f \tag{15}$$

4. Updates voltage angles for all non-slack buses and voltage magnitudes for all `PQ` buses using the correction vector Δx.
5. Terminates when $\max(|f|) <$ `tol` and sets `converged` to `True`. Raises a `ValueError` if convergence is not achieved within `max_iter` iterations.

Returns a dictionary containing the convergence status, iteration count, and final per-unit voltage magnitudes and angles for each bus.

- `solve_fault(fault_bus, vf)`: Executes the symmetrical fault analysis procedure for a bolted three-phase fault at the specified bus. The method proceeds as follows:

1. Constructs the faulted Y_{bus} by calling `calc_ybus_fault()`, which incorporates generator subtransient admittances into the diagonal entries.
2. Inverts the faulted Y_{bus} to obtain Z_{bus} by calling `calc_zbus()`.
3. Extracts the driving-point impedance Z_{nn} from the diagonal entry of Z_{bus} corresponding to the faulted bus.
4. Computes the subtransient fault current as:

$$I''_F = \frac{V_F}{Z_{nn}} \tag{16}$$

where V_F is the pre-fault voltage, defaulting to 1.0 p.u.

5. Computes the post-fault voltage at every bus k as:

$$E_k = 1 - \frac{Z_{kn}}{Z_{nn}} V_F \tag{17}$$

Returns a dictionary containing the faulted bus name, pre-fault voltage, Z_{nn}, fault current, and post-fault bus voltages.

- `print_fault_results(fault_results)`: Formats and prints the fault study results to the console, including the faulted bus, pre-fault voltage, Z_{nn}, subtransient fault current, and the post-fault voltage magnitude at each bus in per-unit.

3.8.3 Role in the Simulator

The `PowerFlow` class unifies the two primary analysis capabilities of the simulator under a single interface. By delegating network assembly, mismatch computation, and matrix operations to the `Circuit` and `Jacobian` classes, the `PowerFlow` class remains focused on algorithmic control flow. The mode-based dispatch through `run_type()` allows the same solver infrastructure to support both steady-state power flow and transient fault analysis without requiring separate entry points, maintaining a clean and extensible design throughout the simulator.

4 Relevant Equations

The following equations are used throughout the simulator to analyze power system behavior.

$$S_{\text{base}} = 100 \text{ MVA} \tag{18}$$

(System Base Apparent Power)

$$p = \frac{\text{MW}_{\text{setpoint}}}{S_{\text{base}}} \tag{19}$$

(Per-Unit Real Power Conversion – Generator)

$$p = \frac{\text{MW}_{\text{load}}}{S_{\text{base}}}, \qquad q = \frac{\text{MVAR}_{\text{load}}}{S_{\text{base}}} \tag{20}$$

(Per-Unit Real and Reactive Power Conversion – Load)

$$Y_{\text{series}} = \frac{1}{r + jx} \tag{21}$$

(Series Admittance – Transformer and Transmission Line)

$$Y_{\text{shunt}} = g + jb \tag{22}$$

(Shunt Admittance – Transmission Line)

$$Y_{\text{prim}}^{\text{transformer}} = \begin{bmatrix} Y_{\text{series}} & -Y_{\text{series}} \\ -Y_{\text{series}} & Y_{\text{series}} \end{bmatrix} \tag{23}$$

(Primitive Admittance Matrix – Transformer)

$$Y_{\text{prim}}^{\text{line}} = \begin{bmatrix} Y_{\text{series}} + \dfrac{Y_{\text{shunt}}}{2} & -Y_{\text{series}} \\ -Y_{\text{series}} & Y_{\text{series}} + \dfrac{Y_{\text{shunt}}}{2} \end{bmatrix} \tag{24}$$

(Primitive Admittance Matrix – Transmission Line Pi-Model)

$$Y_{\text{bus}}[i,i] \mathrel{+}= Y_{\text{prim}}[0,0], \quad Y_{\text{bus}}[i,j] \mathrel{+}= Y_{\text{prim}}[0,1], \quad Y_{\text{bus}}[j,i] \mathrel{+}= Y_{\text{prim}}[1,0], \quad Y_{\text{bus}}[j,j] \mathrel{+}= Y_{\text{prim}}[1,1] \tag{25}$$

(Ybus Stamping – Network Assembly)

$$P_i = |V_i| \sum_{j=1}^{N} |V_j| \left(G_{ij} \cos \delta_{ij} + B_{ij} \sin \delta_{ij}\right) \tag{26}$$

(Real Power Injection)

$$Q_i = |V_i| \sum_{j=1}^{N} |V_j| \left(G_{ij} \sin \delta_{ij} - B_{ij} \cos \delta_{ij}\right) \tag{27}$$

(Reactive Power Injection)

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}, \qquad \Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}} \tag{28}$$

(Power Mismatch)

$$J \cdot \Delta x = f \tag{29}$$

(Newton–Raphson Linear Correction System)

$$J = \begin{bmatrix} J_1 & J_2 \\ J_3 & J_4 \end{bmatrix} = \begin{bmatrix} \dfrac{\partial P}{\partial \delta} & \dfrac{\partial P}{\partial |V|} \\ \dfrac{\partial Q}{\partial \delta} & \dfrac{\partial Q}{\partial |V|} \end{bmatrix} \tag{30}$$

(Jacobian Matrix)

$$J_{1,kk} = -Q_k - B_{kk} V_k^2, \qquad J_{1,kn} = V_k V_n \left(G_{kn} \sin \delta_{kn} - B_{kn} \cos \delta_{kn}\right) \tag{31}$$

(Jacobian Submatrix J_1 – $\partial P / \partial \delta$)

$$J_{2,kk} = \frac{P_k}{V_k} + G_{kk} V_k, \qquad J_{2,kn} = V_k \left(G_{kn} \cos \delta_{kn} + B_{kn} \sin \delta_{kn}\right) \tag{32}$$

(Jacobian Submatrix J_2 – $\partial P / \partial |V|$)

$$J_{3,kk} = P_k - G_{kk} V_k^2, \qquad J_{3,kn} = -V_k V_n \left(G_{kn} \cos \delta_{kn} + B_{kn} \sin \delta_{kn}\right) \tag{33}$$

(Jacobian Submatrix J_3 – $\partial Q / \partial \delta$)

$$J_{4,kk} = \frac{Q_k}{V_k} - B_{kk} V_k, \qquad J_{4,kn} = V_k \left(G_{kn} \sin \delta_{kn} - B_{kn} \cos \delta_{kn}\right) \tag{34}$$

(Jacobian Submatrix J_4 – $\partial Q/\partial |V|$)

$$y'' = \frac{1}{jX''} \tag{35}$$

(Generator Subtransient Admittance)

$$Z_{\text{bus}} = Y_{\text{bus}}^{-1} \tag{36}$$

(Bus Impedance Matrix)

$$I_F'' = \frac{V_F}{Z_{nn}} \tag{37}$$

(Subtransient Fault Current)

$$E_k = 1 - \frac{Z_{kn}}{Z_{nn}} V_F \tag{38}$$

(Post-Fault Bus Voltage)

5 Example Case

5.1 Problem Definition

The goal of this simulator is to perform Newton–Raphson power flow analysis and symmetrical fault analysis on a five-bus power system. The five-bus reference system is taken from the textbook *Power System Analysis and Design* by Glover, Sarma, and Overbye (Case 6.9), which provides a well-documented benchmark with known analytical solutions. The simulator computes per-unit voltage magnitudes and angles at each bus under steady-state conditions, as well as subtransient fault currents and post-fault bus voltages under bolted three-phase fault conditions.

5.2 Solution Process

The solution approach involved implementing the five-bus system by instantiating all network components through the `Circuit` class, including five buses, two transformers, six transmission lines, two generators, and three loads. All parameters were entered in per-unit on a 100 MVA system base consistent with the textbook reference.

For power flow analysis, the `PowerFlow` solver was initialized in `power_flow` mode and executed using the Newton–Raphson iterative algorithm with a convergence tolerance of 1×10^{-3} per-unit and a maximum of 50 iterations. Bus One was designated as the `Slack` bus, Bus Three as a `PV` bus, and Buses Two, Four, and Five as `PQ` buses. A flat start was applied at initialization, with all voltage magnitudes set to 1.0 p.u. and all angles set to 0.0°.

For fault analysis, the `PowerFlow` solver was re-initialized in `fault` mode. The faulted Y_{bus} was constructed by incorporating generator subtransient reactances into the diagonal entries, and the bus impedance matrix Z_{bus} was obtained by matrix inversion. The subtransient fault current and post-fault bus voltages were then computed for a bolted three-phase fault at the specified bus.

5.3 Expected Output and Validation

To validate the simulator, all results were compared against the PowerWorld Simulator solution of the same five-bus system. The PowerWorld one-line diagram, shown in Figure **??**, confirms the reference voltage magnitudes, angles, and power flows used for validation.

The validated power flow results for the five-bus system are as follows:

- Bus One (Slack): $|V| = 1.000$ p.u., $\delta = 0.000^\circ$
- Bus Two (PQ): $|V| = 0.834$ p.u., $\delta = -22.406^\circ$
- Bus Three (PV): $|V| = 1.050$ p.u., $\delta = -0.597^\circ$
- Bus Four (PQ): $|V| = 1.019$ p.u., $\delta = -2.834^\circ$
- Bus Five (PQ): $|V| = 0.974$ p.u., $\delta = -4.548^\circ$

These results are consistent with the PowerWorld solution and the published textbook values, confirming that the Newton–Raphson solver converges to the correct operating point and that the Y_{bus} assembly, power injection computations, and Jacobian construction are all implemented correctly. The simulator was further validated for fault analysis by verifying that the faulted bus voltage collapses to zero for a bolted fault and that the subtransient fault current and remaining bus voltages match the expected theoretical values.

6 Running the Simulator

The simulator is designed to be executed through a main script that defines the power system configuration and calls the appropriate analysis method. A user begins by creating an instance of the `Circuit` class and adding all required system components, including buses, transformers, transmission lines, generators, and loads. Each component is defined using per-unit parameters consistent with the system base.

Once the network is constructed, the user initializes a `PowerFlow` object by passing in the configured `Circuit` instance along with a `Jacobian` object. The desired analysis mode is selected within the `PowerFlow` class, allowing the user to perform either steady-state power flow analysis or symmetrical fault analysis.

For power flow analysis, the solver is executed by calling the `solve()` method. This method automatically assembles the system admittance matrix Y_{bus}, applies a flat start initialization, computes the power mismatch vector, and iteratively updates bus voltage magnitudes and phase angles until convergence is achieved based on the specified tolerance.

For fault analysis, the user specifies the faulted bus and invokes the fault solver, which constructs the modified admittance matrix, computes the bus impedance matrix Z_{bus}, and determines the fault current and post-fault bus voltages.

The results of each analysis are returned as structured dictionaries containing key system values, including bus voltages, phase angles, convergence status, and fault quantities when applicable. These outputs can be printed to the console or further processed for visualization and validation.

This design allows the user to easily modify system parameters, test different operating conditions, and analyze system behavior through a clear and modular interface.